

Description	Keystroke	Function	Note
Run Tests in Batch Mode	To execute tests from the command line, use Emacs in batch mode to load the specific tests and run them using one of the following ERT functions. • For example: <code>emacs -batch -l ert -l my-tests.el -f ert-run-tests-batch-and-exit</code>		
Run batch test	(ert-run-tests-batch &optional SELECTOR)	• Run the tests specified by SELECTOR, printing results to the terminal.	
	• SELECTOR works as described in ‘ert-select-tests’, except if SELECTOR is nil, in which case all tests rather than none will be run; this makes the command line "emacs -batch -l my-tests.el -f ert-run-tests-batch-and-exit" useful. Returns the stats object.		
Run batch tests and exit	(ert-run-tests-batch-and-exit &optional SELECTOR)	Like ‘ert-run-tests-batch’, but exits Emacs when done.	
	• The exit status will be 0 if all test results were as expected, 1 on unexpected results, or 2 if the tool detected an error outside of the tests (e.g. invalid SELECTOR or bug in the code that runs the tests).		
Emacs Lisp Test Code	Test code is written using forms that use the ert-deftest macro. See the following macro descriptions		
Define a test	(ert-deftest NAME () [DOCSTRING] [:expected-result RESULT-TYPE] [:tags '(TAG...)] BODY...)	Define NAME (a symbol) as a test. • BODY is evaluated as a ‘progn’ when the test is run. It should signal a condition on failure or just return if the test passes. • ‘should’, ‘should-not’, ‘should-error’ and ‘skip-unless’ are useful for assertions in BODY. • Use ‘ert’ to run tests interactively. • Tests that are expected to fail can be marked as such using :expected-result. See ‘ert-test-result-type-p’ for a description of valid values for RESULT-TYPE.	
The should macro	(should FORM)	Evaluate FORM. If it returns nil, abort the current test as failed. • Returns the value of FORM.	
The should-not macro	(should-not FORM)	Evaluate FORM. If it returns non-nil, abort the current test as failed. • Returns nil.	
The should-error macro	(should-error FORM &rest KEYS &key TYPE EXCLUDE-SUBTYPES)	Evaluate FORM and check that it signals an error.	
	• The error signaled needs to match TYPE. TYPE should be a list of condition names. (It can also be a non-nil symbol, which is equivalent to a singleton list containing that symbol.) If EXCLUDE-SUBTYPES is nil, the error matches TYPE if one of its condition names is an element of TYPE. If EXCLUDE-SUBTYPES is non-nil, the error matches TYPE if it is an element of TYPE. • If the error matches, returns (ERROR-SYMBOL . DATA) from the error. If not, or if no error was signaled, abort the test as failed.		
The skip-unless macro	(skip-unless CONDITION)	Skip the current ert-deftest defined test unless CONDITION is non-nil. • Normally used to skip tests when the environment does not provide the necessary conditions for a valid test.	
The :expected-result tag	:expected-result • :failed • :passed	Tag tests of lesser importances that are expected to fail potentially under some conditions. It can also be used to silence the report of a bug while leaving in the test. • See the documentation .	
Test Coverage highlighting	To measure test coverage of elisp code in a file first activate the coverage and then run the tests that exercise the code. After the test you can then activate marking of areas that have not been tested.		
Identify file to mark code being use during upcoming test.	<f12> C-t C-c C-s	(testcover-start FILENAME &optional BYTE-COMPILE)	Prompt for FILENAME, then use Edebug to instrument for coverage all macros and functions inside it. • If BYTE-COMPILE is non-nil, byte compile each function after instrumenting.
Start test coverage for current function	<f12> C-t C-c .	(testcover-this-defun)	Start coverage on function under point.
Overlays all forms that have low test coverage	<f12> C-t C-c C-a	(testcover-mark-all &optional BUFFER)	Mark all forms in BUFFER that did not get completely tested during coverage tests. • This function creates many overlays in the buffer.
Remove coverage overlays	<f12> C-t C-c C-u	(testcover-unmark-all BUFFER)	Remove all overlays from FILENAME.
Move point to next line with low coverage.	<f12> C-t C-c C-n	(testcover-next-mark)	Move point to next line in current buffer that has a splotch.
Toggle global minor mode to mark whole lines	<f12> C-t C-c C-l	(testcover-mark-line-mode &optional ARG)	Toggle global minor mode to mark whole line with testcover. 📦 Requires test-cover-mark activated by  pel-use-testcover-mark
Extend the test cover overlays to eat entire line	<f12> C-t C-c C-m	(testcover-mark-line-mark-all)	Mark all forms that didn’t get completely tested, with lines, extending the overlays that are created by test cover itself. 📦 Requires test-cover-mark activated by  pel-use-testcover-mark
End coverage instrumentation.	<f12> C-t C-c C-e	(testcover-end FILENAME)	Turn off instrumentation of all macros and functions in FILENAME.

Emacs Lisp Testing — References

Topic & Link	Description
ERT Manuals	
ERT : Emacs Lisp Regression Testing	ERT Manual, part of Emacs.
Test Coverage — Emacs Lisp	Test coverage section of the GNU Emacs Lisp manual
ERT Tools Repositories and Files	
pel-ert.el	pel-ert.el defines a set of equality predicates that accept extra arguments for the sole purpose of having them shown in the ERT report of a failed test. To see a set of <i>test environment</i> variables in the test report just pass them as extra arguments to these equality predicates.
testcover.el source	Source of the test coverage support
overseer.el @ GitHub	
ert-runner @ GitHub	
Emacs Lisp Mock @ EmacsWiki	The original location of that mock library that can be used with ert.
El mock @ GitHub	The new location for el-mock.el
emacs-noflet @ GitHub	External package that provides the noflet macro which can be used to re-define functions locally. Can be used in ERT testing. I created my own fork of this and integrated most long standing pull requests. See pierre-rouleau / emacs-noflet .
Articles/Blogs on ERT	Interesting articles to read before writing Emacs Lisp ERT testing code.
Quick intro to ert testing	A quick overview of ERT based Emacs Lisp testing.
Elisp Unit Testing with ERT	Quick overview of ERT in an August 2012 blog written by Chris Wellons. Since then the cl.el library was replaced by the cl-lib.el and the flet function was deprecated to cl-flet, but aside from these small items the description is still valid.
flet, cl-flet, cl-letf and noflet	Make Flet Great Again : Another great post from Chris Wellons. - Describes the new cl-flet and cl-letf, and how to use cl-letf to create Ert test that modify the behaviour of called functions. Understanding letf and how it replaces flet, by Arthur Malabarba - Describes cl-letf and how it can replace the old flet.
ERT: Emacs Lisp Regression Testing	A nice description of the ERT features and techniques given at the NYC meetup. Describes: - How to write a simple test, how to run the test and use the *ert* buffer commands. - How to test expected errors with should-error
Continuous integration and code coverage for Emacs packages with Travis and Coveralls	An overview of automated Emacs Lisp unit testing with coverage measurement from Sacha Chua blog.
Reddit discussion: Testing in Emacs	Interesting discussion on testing under Emacs. Leads to several interesting projects: test-cockpit for testing code written in several programming languages under Emacs dape : Debug Adapter Protocol for Emacs stan-mode which uses buttercup for testing.
Other Unit Testing Support	Other packages and libraries for testing are not listed at the top of this page and currently not explicitly supported by PEL. Some are listed here.
Unit Testing on EmacsWiki	Provides a list of alternatives to ERT with other testing-related information.
emacs-test-simple @ GitHub	An Alternative to ERT developed by debugger expert R.Bernstein (see also here).
undercover.el	A test coverage library for Emacs. Does not support byte-compilation nor circular objects. Meant to be used with Cask, Eask or Eldev.
Shell based unit test tools	
cram	A simple test system to test shell commands. cram is a command line tool that takes cram scripts stored in .t files. PEL supports the cram-mode when pel-use-cram-mode is turned on.
Emacs Lisp Concepts	Several Emacs Lisp concepts are useful when writing ERT test and mock ups.
Scoping Rules for Variable Bindings	When writing Emacs Lisp code and test with mockup in particular its important to fully understand the concepts of dynamic and lexical binding in Emacs Lisp.  Be aware that starting with <u>version 27 of Emacs Lisp lexical binding is the default</u> while dynamic binding was the default in previous versions.
Github Continuous Integration	
With Github Actions	<u>Workflow syntax for GitHub Actions</u> <u>Github Actions Runner Images</u> : select the os environment <u>Setup Emacs for Github Actions</u> : select the Emacs version
Running ERT tests suite with Cask and ert-runner	- To run all tests: <code>cask exec ert-runner</code> - To run a specific test: <code>cask exec ert-runner -p "name-of-specific-test"</code> - To run tests that are tagged fast or important: <code>cask exec ert-runner -t fast,important</code> - To run tests not tagged 'network': <code>cask exec ert-runner -t '! network'</code> - Combining options: <code>cask exec ert-runner test/moda-test.el -p "specific-functionality"</code>